

GAS OPTIMIZATION :

Unoptimized gas report (forge test--gas-report) :

src/Auction.sol:Auction Contract					
Deployment Cost	Deployment Size				
1677881	7205				
Function Name	Min	Avg	Median	Max	# Calls
buyerFee	7371	7371	7371	7371	7
createAuction	236748	236748	236748	236748	5
endAuction	35427	48771	48771	62116	2
getAuction	18165	18165	18165	18165	1
placeBid	46884	99948	111795	111795	7
registerAsSeller	91390	91390	91390	91390	5
verifySeller	30503	30503	30503	30503	5
withdrawBid	38281	38281	38281	38281	2

Highest gas consuming function : createAuction()

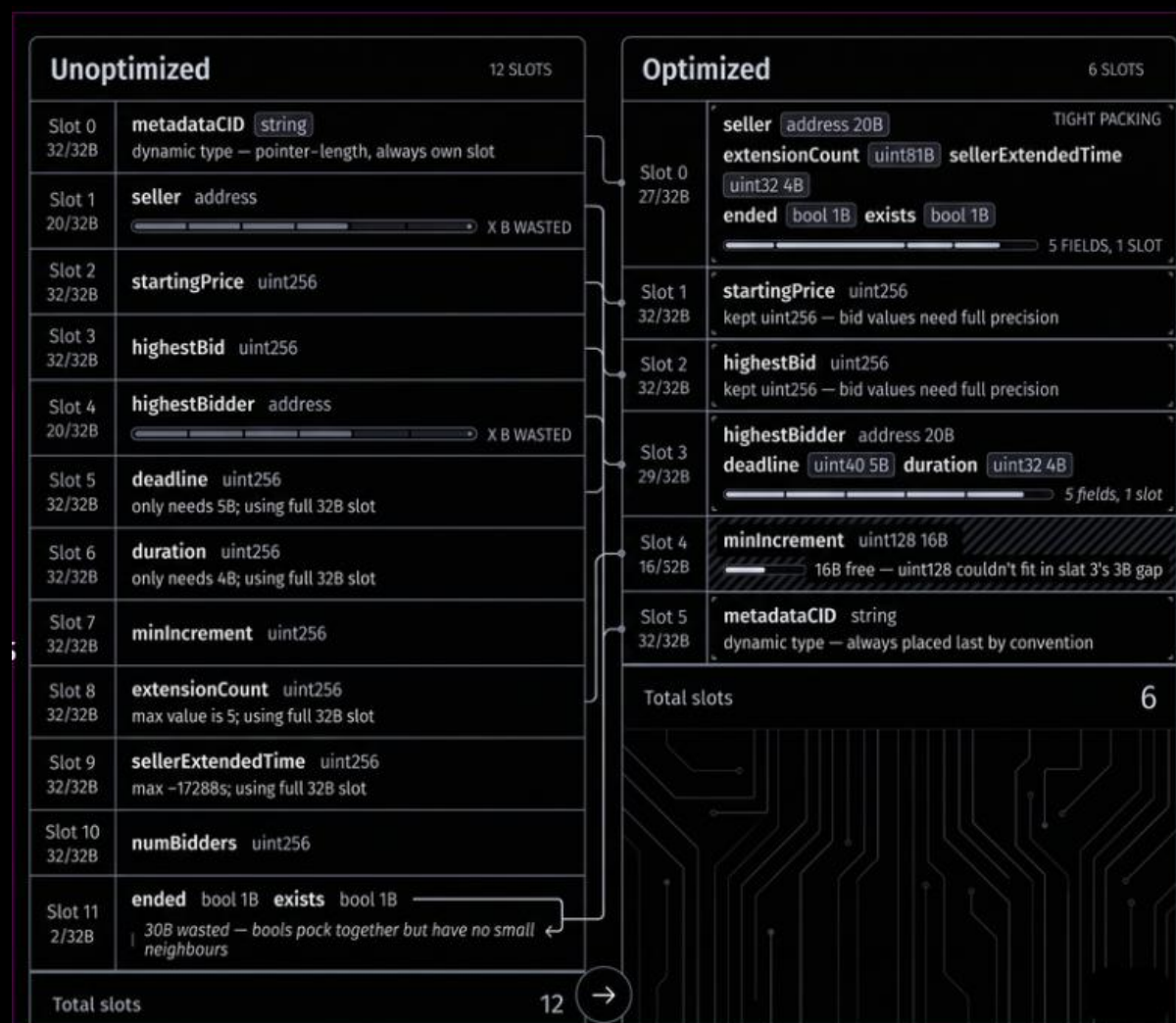
Analysis - why it consumed max gas :

EVM manages storage in 32-byte slots. Each time a new auction is initialized, the contract uses the SSTORE opcode-the most expensive operation in Solidity to write data permanently to the blockchain.

The EVM fills these storage slots in the exact order that variables are declared in the code. If small variables (like bool or address) are separated by large 32-byte variables (like uint256), the EVM cannot group them together. Instead, it is forced to waste the unused space in the current slot and start a new 32-byte slot for the next variable. Since each new slot costs 20,000 gas to initialize, this gap in storage significantly increases the total gas cost of the function.

Solution : Variable packing

This involves organizing the struct to maximize storage efficiency. By using downcasting—choosing smaller data types like uint64 or uint8 for variables like duration or extensionCount that are guaranteed to fit in less space—we grouped multiple variables together within a single 32-byte slot. This strategic ordering allowed the EVM to consolidate several pieces of data into one SSTORE operation instead of multiple expensive ones. Ultimately, this approach ensured that the contract uses the minimum number of storage slots possible, significantly lowering the gas cost for every new auction created.



Other Optimizations :

- **Calldata vs Memory :**

In the original contract, dynamic inputs like string itemName were stored in memory. This caused the EVM to copy data from calldata into memory, increasing gas usage.

The contract was updated to use calldata for external function parameters. Since calldata is read-only and already holds the input data, this removes the need for copying and reduces gas costs for functions like createAuction().

- **Custom Errors :**

require statements with long error messages were replaced with custom errors.

Lower deployment cost: Long strings increased contract size, making deployment more expensive.

Lower execution cost: Custom errors used a short 4-byte selector instead of full strings, saving gas when transactions revert.

Overall, these changes reduced both deployment and execution gas costs.

Optimized gas report (forge test--gas-report) :

src/Auction.sol:Auction Contract					
Deployment Cost	Deployment Size				
1458793	6187				
Function Name	Min	Avg	Median	Max	# Calls
buyerFee	7311	7311	7311	7311	7
createAuction	186080	186080	186080	186080	5
endAuction	33253	44482	44482	55711	2
getAuction	11913	11913	11913	11913	1
placeBid	40544	83714	88206	96317	7
registerAsSeller	91390	91390	91390	91390	5
verifySeller	30497	30497	30497	30497	5
withdrawBid	38175	38175	38175	38175	2

- Overall deployment cost reduced from 1677881 to 1458791, saved 219090
- createAuction() cost reduced from 236748 to 186080 saved 50668